GUIDA AL PROGETTO

Market Regime Explorer

Costruire un modello che descriva i regimi di mercato e una dashboard per esplorarne probabilità, incertezza e comportamento dei tre asset. Il team scrive il codice e consegna una pipeline riproducibile.

8-12	3.868	1 CSV
settimane	date comuni	19 colonne

Il dataset originale

data/market_data.csv contiene dati giornalieri dal **18 novembre 2004 al 1 aprile 2020**. Le tre serie sono già unite sulle date comuni, senza riempimenti o modifiche ai valori. Nessuna feature è già calcolata.

ASSET	ESPOSIZIONE	PREZZO
SPY	Azioni USA / S&P 500	spy_adj_close
TLT	Treasury USA a lunga scadenza	tlt_adj_close
GLD	Oro tramite ETF	gld_adj_close

Prezzi in USD. Le altre colonne sono OHLC e volume per asset. I rendimenti da prezzi aggiustati sono proxy dei rendimenti aggiustati; non mescolare queste serie con OHLC non aggiustati. Il campione include il 2008 e l'inizio del COVID, ma non la ripresa o il 2022.

Cosa costruire

15 feature candidate: rendimento, volatilità, momentum e drawdown per ciascun asset, più tre correlazioni. Finestre iniziali: 20 rendimenti per volatilità, 60 intervalli per momentum, 252 prezzi per drawdown, 60 rendimenti per correlazioni. Selezionare circa 5-8 input sul training.

K-Means → GMM → HMM. Confrontare 2-4 stati e tre seed; usare covarianze diagonali per GMM e HMM. Massimo 40 tentativi HMM complessivi. Dare un nome finanziario agli stati dopo averne letto i profili, senza forzare tre fasi prestabilite.

Lo starter contiene celle vuote e interfacce da implementare. Nessun modello allenato o risultato finale è fornito. Librerie gratuite, Python 3.12 e laptop locale; nessuna API live durante l'esecuzione.

IL PERCORSO

Dal notebook all’app

Aprire i notebook in ordine. Ogni passaggio indica cosa costruire e verificare, in italiano e inglese, prima della cella vuota. Le brevi note iniziali richiamano soltanto le nozioni utili per quella fase.

FASE	LAVORO E CONSEGNA
01	**Dati e feature** Leggere il CSV, verificare date e valori, costruire EDA e feature. Testare che modifiche al futuro non cambino le feature passate. Output: features.csv; grafici EDA.
02	**Selezione e baseline** Dividere i dati, selezionare gli input e stimare lo scaler sul training. Allenare K-Means e GMM; confrontare configurazioni e profili. Output: baseline_comparison.csv; preprocessing.
03	**HMM e validazione** Allenare, scegliere i parametri, implementare il filtro e misurare stabilità, incertezza e profili dei tre asset. Proiettare gli stati con pi_t @ A. Output: diagnostiche, probabilità, profili e confronti.
04	**Pipeline e dashboard** Collegare i moduli, salvare il bundle, scrivere i test e costruire l'app. Esportare figure coerenti con la dashboard e provare un ambiente pulito. Output: pipeline, app e figure.

Una struttura essenziale

FILE / CARTELLA	USO
data/market_data.csv	Unico input, da conservare locale.
settings.json	Finestre, split e parametri iniziali.
core/	Funzioni condivise: dati, feature, selezione, modelli, forecast, validazione, grafici e pipeline.
tests/ · results/	Test da scrivere; tutti gli output da generare.
run.py · app.py	Avvio della pipeline e dashboard da implementare.

Sviluppare, verificare, integrare

Provare una funzione nel notebook, trasferirla in **core/**, aggiungere un test utile e richiamarla dal notebook. Ogni parte deve avere una piccola consegna eseguibile. Assegnare responsabili e revisori in base alle disponibilità, senza gruppi prefissati.

VALUTAZIONE E CONSEGNA

Un modello da spiegare

TRAIN 60%	VALIDATION 20%	TEST 20%

Ordine cronologico, dopo il warm-up

Training: selezione, scaler e fit. **Validation:** scelta dei parametri. **Test:** valutazione con scelte congelate. Confrontare likelihood solo sugli stessi input e scaler. Le probabilità fuori campione usano un modello fisso e il filtro continua tra i blocchi; i grafici del training e lo smoothing restano retrospettivi.

Misurare occupazione e durata degli stati, cambi di regime, stabilità tra seed ed entropia delle probabilità. Leggere insieme rendimenti, volatilità, drawdown e correlazioni dei tre asset. I regimi sono stati latenti del modello, non etichette di mercato osservate.

Previsione: un’estensione contenuta

La proiezione **pi_t @ A** stima gli stati alla prossima seduta. In seguito, provare il rischio SPY a cinque sedute: confrontare prevalenza, logit sulla volatilità corrente e stessa logit con probabilità HMM. Soglia sul training, target entro ogni blocco, parametri sulla validation; Brier, average precision e controllo su finestre non sovrapposte. Nessun vantaggio predittivo è garantito.

8-12 settimane, consegne progressive

SETTIMANE	RISULTATO
1-2	CSV validato, EDA e primi test.
3-4	Feature, selezione, scaler e baseline.
5-7	HMM, filtro, profili e confronto dei modelli.
8-10	Pipeline, dashboard, figure e consegna.
11-12	Buffer e forecast opzionale; app in parallelo.

Consegna pronta quando...

python -m pytest, **python run.py** e **streamlit run app.py** funzionano su un altro laptop. L'app legge gli output salvati, gestisce file mancanti e non riaddestra al cambio dei controlli. Consegnare timeline, probabilità e incertezza, profili dei tre asset e confronto dei modelli; esportare 3-4 figure leggibili.

Dati e pubblicazione. Il CSV locale e gli output che riproducono quotazioni sono esclusi da Git. CC0 dichiarata su Kaggle; diritti della fonte non verificati. L'uso accademico e la diffusione dei risultati richiedono una base applicabile. Il campione è già stato esaminato durante la preparazione: il test è una valutazione storica didattica, non una nuova verifica indipendente.

Fonte: Stock Market Dataset, Oleh Onyshchak (jacksoncrow), v2; origine Yahoo Finance/yfinance. Restrizioni Yahoo. Provenienza e hash del CSV nel README.

PROJECT GUIDE

Market Regime Explorer

Build a model of market regimes and a dashboard for exploring their probabilities, uncertainty and three-asset behaviour. The team writes the code and delivers a reproducible pipeline.

8-12	3,868	1 CSV
weeks	common dates	19 columns

The original dataset

data/market_data.csv contains daily data from **18 November 2004 to 1 April 2020**. The three series are already joined on common dates, without filling or changing values. No features are precomputed.

ASSET	EXPOSURE	PRICE
SPY	US equities / S&P 500	spy_adj_close
TLT	Long-term US Treasuries	tlt_adj_close
GLD	Gold through an ETF	gld_adj_close

Prices are in USD. Other columns are OHLC and volume per asset. Returns from adjusted prices are adjusted-return proxies; do not mix these series with unadjusted OHLC. The sample includes 2008 and early COVID, but not the recovery or 2022.

What to build

15 candidate features: return, volatility, momentum and drawdown per asset, plus three correlations. Starting windows: 20 returns for volatility, 60 intervals for momentum, 252 prices for drawdown and 60 returns for correlations. Select about 5-8 inputs on training data.

K-Means → GMM → HMM. Compare 2-4 states and three seeds; use diagonal covariances for GMM and HMM. At most 40 total HMM fits. Assign financial names after inspecting state profiles, without forcing three predetermined phases.

The starter contains empty cells and interfaces to implement. No trained model or final results are supplied. Free libraries, Python 3.12 and a local laptop; no live API during execution.

THE WORKFLOW

From notebook to app

Open the notebooks in order. Each step states what to build and check, in Italian and English, before an empty cell. Short opening notes cover only the concepts needed for that stage.

STEP	BUILD AND DELIVER
01	**Data and features** Read the CSV, validate dates and values, build EDA and features. Test that future changes do not alter earlier features. Output: features.csv; EDA charts.
02	**Selection and baselines** Split the data, select inputs and fit scaling on training. Fit K-Means and GMM; compare configurations and profiles. Output: baseline_comparison.csv; preprocessing.
03	**HMM and validation** Fit models, choose parameters, implement filtering and measure stability, uncertainty and three-asset profiles. Project states with pi_t @ A. Output: diagnostics, probabilities, profiles and comparisons.
04	**Pipeline and dashboard** Connect modules, save the bundle, write tests and build the app. Export charts consistent with the dashboard and check a clean environment. Output: pipeline, app and figures.

A minimal structure

FILE / FOLDER	PURPOSE
data/market_data.csv	Single input, kept local.
settings.json	Windows, splits and starting parameters.
core/	Shared functions: data, features, selection, models, forecasting, validation, charts and pipeline.
tests/ · results/	Tests to write; all outputs to generate.
run.py · app.py	Pipeline launcher and dashboard to implement.

Develop, check, integrate

Develop a function in the notebook, move it into **core/**, add a useful test and call it from the notebook. Each part needs a small runnable delivery. Assign owners and reviewers according to availability, without fixed groups.

EVALUATION AND RELEASE

A model you can explain



Chronological order, after warm-up

Training: selection, scaling and fitting. **Validation:** parameter choice. **Test:** evaluation with frozen choices. Compare likelihood only on identical inputs and scaling. Out-of-sample probabilities use a fixed model and filtering continues across blocks; training charts and smoothing remain retrospective.

Measure state occupancy and duration, switching, seed stability and probability entropy. Inspect returns, volatility, drawdowns and correlations across the three assets. Regimes are latent model states, not observed market labels.

Forecasting: a contained extension

The **pi_t @ A** projection estimates next-session states. Then, optionally test five-session SPY risk: compare prevalence, a logit on current volatility and the same logit with HMM probabilities. Set the threshold on training, keep targets within each block and tune on validation; use Brier, average precision and a non-overlapping-window check. Predictive improvement is not guaranteed.

8-12 weeks, progressive delivery

WEEKS	DELIVERY
1-2	Validated CSV, EDA and first tests.
3-4	Features, selection, scaling and baselines.
5-7	HMM, filtering, profiles and model comparison.
8-10	Pipeline, dashboard, figures and release.
11-12	Buffer and optional risk forecast; app work in parallel.

Ready to ship when...

python -m pytest, **python run.py** and **streamlit run app.py** work on another laptop. The app reads saved outputs, handles missing files and does not retrain when controls change. Deliver timelines, probabilities and uncertainty, three-asset profiles and model comparison; export 3-4 readable figures.

Data and publication. The local CSV and outputs reproducing quotes are excluded from Git. Kaggle declares CC0; upstream rights have not been verified. Academic use and publication of results require an applicable legal basis. The sample was inspected during preparation: the test is an educational historical evaluation, not a new independent verification.

Source: Stock Market Dataset, Oleh Onyshchak (jacksoncrow), v2; upstream Yahoo Finance/yfinance. Yahoo restrictions. CSV provenance and hash are in the README.